Sample Java program to check whether the given number is prime palindrome or not.

Prime Palindrome Program

1. Algorithm

Algorithm to Check Prime Palindrome

Step 1: Start the program.

Step 2: Read an integer num from the user.

Step 3: Check whether the number is negative or zero.

- If num < 0 or num == 0, display **Invalid** and stop the checking process.

Step 4: Check whether num == 1.

- If yes, display **The number is not a Prime Palindrome** and stop.

Step 5: Assume the number is prime by setting x = true.

Step 6: Check whether the number is divisible by any number from 2 to num-1.

- If num % i == 0, the number is not prime.
- Set x = false and stop the loop.

Step 7: If x is true, call the check() method to check whether the number is a palindrome.

Step 8: In the check() method:

1. Store the original number in real.
2. Set rev = 0.
3. Extract the last digit using n % 10.
4. Add the digit to rev.
5. Remove the last digit using n / 10.
6. Repeat until n becomes zero.

Step 9: Compare rev with real.

- If rev == real, display **The given number is a prime palindrome**.
- Otherwise, display **The number is prime but not a palindrome**.

Step 10: Stop the program.

2. Execution Steps

Step 1: Create the Java Class

Create a class named:

PrimePalindrome

inside the observation package.

Step 2: Run the Program

Run the program using:

Run → Run As → Java Application

Step 3: Enter a Number

The program asks:

Enter the number you wanna check for a primepalindrome

For example:

Enter the number you wanna check for a primepalindrome

131

Step 4: Prime Checking

The program checks whether 131 is divisible by any number from 2 to 130.

Since it is not divisible by any of them, 131 is prime.

Step 5: Palindrome Checking

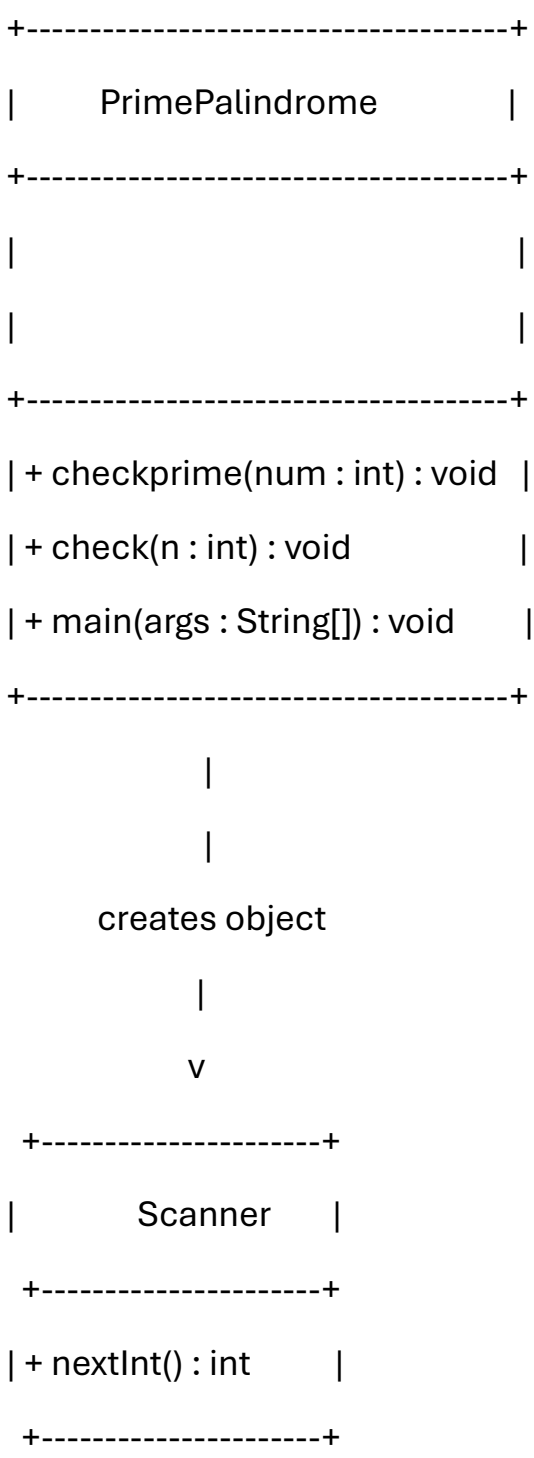
The digits of 131 are reversed:

Original number = 131

Reversed number = 131

Since both are equal, the number is a palindrome.

3. UML Class Diagram



Source Code:

```
package observation;

import java.util.Scanner;
```

```
public class PrimePalindrome {
	void checkprime(int num) {
		boolean x=true;
		if(num<0 || num==0) {
			System.out.println("the check for ZER or negative INVALID");
		}
		else if(num==1) {
		  System.out.println("The number is not a Prime Palindrome");
		  return;
		}

		else {
		for(int i=2;i<num;i++) {
			if(num%i==0) {
				System.out.println("the number is not a primepalindrome");
				x=false;
				break;
			}
		}

		}
		if(x) {
			check(num);
		}
	}
	void check(int n) {
		int real=n;
		int rev=0,r;
```

```
            while(n>0) {
            r=n%10;
            n=n/10;
            rev=rev*10+r;
            }
            if(rev==real) {
                System.out.println("the given number is a primepalindrome");
            }
            else {
                System.out.println("tehe number is prime but not a palindrome");
            }
        }
public static void main(String[] args) {
        int number;
        Scanner sc=new Scanner(System.in);
        System.out.println("enter the number you wanna check for a primepalindrome");
        number =sc.nextInt();
        PrimePalindrome p=new PrimePalindrome();
        p.checkprime(number);
}
}
```

Output: